
SECURE-DROP: A ZERO-RELAY, END-TO-END EN-CRYPTED PEER-TO-PEER FILE TRANSFER SYSTEM

Project Report Submitted in Partial Fulfilment of the Requirements for the Degree of
Bachelor of Technology

in

Computer Science and Engineering

Submitted by

Name of the Student 1: *(Roll No. _____)* Name of the Student 2: *(Roll No. _____)* Name
of the Student 3: *(Roll No. _____)*

Under the Supervision of

*(Name of Project/Faculty Advisor) (Designation) (Department of Computer Science and
Engineering)*

DIT University, Dehradun

May, 2026

DECLARATION

I/We declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed. The plagiarism check report is attached at the end of this document.

Name of the Student _____ Signature and Date _____

Name of the Student _____ Signature and Date _____

Name of the Student _____ Signature and Date _____

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to our project advisor for the continuous guidance, encouragement, and invaluable feedback throughout the course of this capstone project. Their expertise in computer networks and information security greatly shaped the design and implementation of Secure-Drop.

We are also thankful to the Department of Computer Science and Engineering, DIT University, for providing the academic framework and resources that made this project possible.

Finally, we acknowledge the open-source communities behind the WebRTC specification, the Python websockets library, the React framework, and the Web Crypto API, whose work forms the technical foundation of this system.

TABLE OF CONTENTS

LIST OF TABLES

Table No.	Title	Page
Table 3.1	Technical Feasibility Assessment	10
Table 3.2	Cost Breakdown (Free-Tier Deployment)	11
Table 4.1	Signaling Message Types and Descriptions	14
Table 4.2	Cryptographic Algorithms and Standards	16
Table 4.3	WebRTC ICE Server Configuration	17
Table 4.4	File Transfer Frame Header Structure	18
Table 5.1	Development Tools and Versions	22
Table 5.2	Frontend Dependencies	22
Table 5.3	Backend Dependencies	23
Table 6.1	Functional Test Cases and Results	33
Table 6.2	Transfer Speed Measurements (LAN)	35
Table 6.3	Transfer Speed Measurements (Internet)	35
Table 6.4	CPU Usage Measurements	36
Table 6.5	Cross-Platform Browser Compatibility	37

LIST OF FIGURES

Figure No.	Title	Page
Fig. 4.1	High-Level System Architecture	12
Fig. 4.2	Signaling Server State Machine	14
Fig. 4.3	ECDH Key Exchange Protocol Flow	15
Fig. 4.4	HKDF Key Derivation Process	16
Fig. 4.5	AES-256-GCM Per-Chunk Encryption	17
Fig. 4.6	Frontend Component Tree	19
Fig. 4.7	Data Flow Diagram — File Transfer	20
Fig. 4.8	Data Flow Diagram — Key Exchange	21
Fig. 5.1	Secure-Drop — Main Interface (Dark Mode)	30
Fig. 5.2	File Zone with File Selected	31
Fig. 5.3	Incoming Transfer Request Modal	31
Fig. 5.4	Active Transfer Progress View	32
Fig. 5.5	QR Code Join Panel	32
Fig. 6.1	Transfer Speed vs. File Size (LAN)	35
Fig. 6.2	CPU Usage Over Transfer Duration	36

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AES	Advanced Encryption Standard
API	Application Programming Interface
CPU	Central Processing Unit
CSE	Computer Science and Engineering
CSS	Cascading Style Sheets
DataChannel	WebRTC DataChannel
DTLS	Datagram Transport Layer Security
E2E	End-to-End
ECDH	Elliptic Curve Diffie-Hellman
GCM	Galois/Counter Mode
HKDF	HMAC-based Key Derivation Function
HMAC	Hash-based Message Authentication Code
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
ICE	Interactive Connectivity Establishment
IV	Initialisation Vector
JSON	JavaScript Object Notation
LAN	Local Area Network
mDNS	Multicast Domain Name System
MITM	Man-In-The-Middle
NAT	Network Address Translation
NIST	National Institute of Standards and Technology
P2P	Peer-to-Peer
RFC	Request for Comments
SDP	Session Description Protocol
SHA	Secure Hash Algorithm
SRTP	Secure Real-Time Transport Protocol
STUN	Session Traversal Utilities for NAT
TLS	Transport Layer Security
TURN	Traversal Using Relays around NAT
WebRTC	Web Real-Time Communication

ABSTRACT

The widespread reliance on cloud-based file sharing services raises significant privacy and security concerns, as files are uploaded to third-party servers where they may be stored, scanned, or intercepted. This project presents **Secure-Drop**, a browser-native, peer-to-peer (P2P) file transfer system that eliminates the need for any cloud intermediary in the data path. Files are encrypted on the sender’s device using **AES-256-GCM** prior to transmission and are transferred directly to the recipient via a **WebRTC DataChannel**. A lightweight Python WebSocket signaling server facilitates connection bootstrapping (SDP/ICE exchange) but never handles file content.

The cryptographic protocol employs **Elliptic Curve Diffie-Hellman (ECDH) P-256** for ephemeral session key exchange, **HKDF-SHA-256** for key derivation, and **SHA-256** for post-transfer integrity verification. A session fingerprint derived from the raw ECDH shared secret enables both parties to detect man-in-the-middle (MITM) attacks by reading an 8-character hex code aloud.

The system is implemented as a **React 18 / TypeScript** single-page application with a **sci-fi inspired** user interface supporting dark and light modes, real-time transfer statistics, drag-and-drop file selection, and QR-code-based device pairing. The application is fully platform-independent, running on any modern browser across macOS, Windows, Linux, iOS, and Android. Testing results demonstrate LAN transfer speeds of 20–80 MB/s, CPU usage below 25% during transfers, and verified compatibility across Chrome, Safari, Firefox, and Edge.

Keywords: Peer-to-Peer File Transfer, End-to-End Encryption, WebRTC, AES-256-GCM, ECDH, Browser Security, Privacy.

CHAPTER 1: INTRODUCTION

1.1 Background

The proliferation of digital content and the need for rapid file sharing have made cloud-based transfer services ubiquitous. Services such as Google Drive, WeTransfer, Dropbox, and iCloud have become the de facto standard for sharing files between individuals and organisations. However, these platforms fundamentally require files to transit through and reside on third-party servers, introducing a chain of custody that users have no control over.

Several high-profile data breaches and revelations about the extent of government surveillance programmes (as documented in academic literature [1]) have heightened awareness of the risks associated with cloud storage. In a cloud-based transfer model, the service

provider has technical access to the plaintext content of files unless client-side encryption is implemented by the user themselves — a step most users do not take.

Web Real-Time Communication (WebRTC), standardised by the World Wide Web Consortium (W3C) and the Internet Engineering Task Force (IETF) [2], offers a compelling alternative. Originally designed for low-latency audio and video communication, WebRTC's DataChannel API [3] provides a general-purpose, browser-native binary data channel between two peers. This channel is encrypted using Datagram Transport Layer Security (DTLS) at the transport layer, and can be augmented with application-level encryption to achieve defence-in-depth.

The Web Crypto API [4], natively available in all modern browsers, exposes hardware-accelerated cryptographic primitives including ECDH key exchange, HKDF key derivation, and AES-GCM symmetric encryption. Combining WebRTC with the Web Crypto API enables a fully browser-native, end-to-end encrypted P2P file transfer system requiring no plugins, native applications, or cloud intermediaries.

1.2 Problem Statement

Existing mainstream file-sharing solutions present the following limitations:

1. **Privacy Exposure:** Files are stored in plaintext on third-party servers, accessible to service providers and potentially law enforcement without user knowledge.
2. **Platform Lock-in:** Apple's AirDrop operates exclusively within the Apple ecosystem; Google's Nearby Share is limited to Android. Cross-platform sharing requires cloud upload.
3. **File Size Limits:** Free tiers of services such as WeTransfer limit transfers to 2 GB; Google Drive imposes 5 TB quotas. WebRTC DataChannel is limited only by available RAM and storage.
4. **Account Requirement:** Most services require registration, creating a digital trail linking users to transferred files.
5. **Network Dependency:** Cloud-based services require internet connectivity even for transfers between devices on the same local network, introducing unnecessary latency and bandwidth consumption.

The absence of a platform-independent, zero-account, end-to-end encrypted, direct P2P file transfer tool accessible through a standard web browser represents a gap that this project addresses.

1.3 Objectives

The primary objectives of this project are:

1. To design and implement a browser-native P2P file transfer system using WebRTC DataChannel that requires no cloud intermediary for file data.
2. To implement a layered end-to-end encryption protocol using ECDH P-256, HKDF-SHA-256, and AES-256-GCM to ensure confidentiality of file content.
3. To provide a MITM detection mechanism through session fingerprinting derived from the ECDH shared secret.
4. To build a platform-independent web application compatible with all major browsers and operating systems.
5. To create an intuitive, accessible user interface with drag-and-drop file transfer, QR-code device pairing, and real-time transfer monitoring.
6. To deploy the application with a free-tier infrastructure (Netlify frontend, Railway signaling server) for global accessibility.

1.4 Scope of the Project

The scope of Secure-Drop encompasses:

- **In scope:** Browser-to-browser file transfer over LAN and the internet; end-to-end encryption at the application layer; WebRTC-based direct P2P connection; Python WebSocket signaling server; web-based user interface; LAN and internet deployment modes.
- **Out of scope:** Native mobile applications; server-side file storage or caching; multi-file batch transfer in a single session; video/audio streaming; user authentication and account management.

1.5 Organisation of the Report

The remainder of this report is organised as follows. Chapter 2 surveys related work in the areas of P2P file transfer, WebRTC applications, and browser-based encryption. Chapter 3 presents a feasibility study covering technical, operational, and economic dimensions. Chapter 4 describes the system architecture and design. Chapter 5 details the implementation of all system components. Chapter 6 presents testing results. Chapter 7 summarises conclusions and Chapter 8 outlines directions for future work.

CHAPTER 2: RELATED WORK / LITERATURE SURVEY

2.1 Cloud-Based File Transfer

The dominant paradigm for file sharing over the past decade has been cloud-centric. Dropbox [5], introduced in 2008, popularised the model of a synchronised folder backed by

cloud storage. Google Drive [6] extended this model with real-time collaborative editing. WeTransfer [7] targeted one-shot large file sharing with a link-based delivery model.

A common architectural characteristic of these systems is the store-and-forward model: the sender uploads the file to a server, which the recipient then downloads. This model provides reliability (the sender need not be online when the recipient downloads) but at the cost of a trusted third party in the data path. Chen et al. [8] analysed cloud storage services and found that a majority did not implement client-side encryption by default, leaving files accessible to service providers and potentially vulnerable to server-side breaches.

2.2 Peer-to-Peer File Transfer Protocols

Early P2P file transfer protocols such as BitTorrent [9] achieved high-throughput distributed file sharing but were designed for public content distribution, not private one-to-one transfers. BitTorrent does not provide built-in encryption for file content, though extensions such as BitTorrent Message Stream Encryption (MSE) [10] provide obfuscation at the transport layer.

Direct Connect (DC++) and its successors provided LAN-based P2P sharing within private networks. However, these protocols required dedicated desktop clients and did not operate in the browser. The emergence of WebRTC changed this landscape fundamentally, enabling browser-native P2P communication without plugins.

2.3 WebRTC for Browser-Based P2P

WebRTC was published as a W3C Candidate Recommendation in 2011 and achieved broad browser support by 2017. The RTCPeerConnection API [2] establishes a direct connection between two browser instances, negotiating network paths through ICE (Interactive Connectivity Establishment) [11] and encoding session parameters via SDP [12].

The RTCDataChannel API [3] provides an unreliable or reliable binary data channel over SCTP/DTLS, suitable for file transfer. Gudipati et al. [13] demonstrated that WebRTC DataChannel achieves transfer rates of up to 500 Mbps in controlled LAN environments. Nurminen et al. [14] studied WebRTC signaling architectures and found that a centralised WebSocket relay for SDP/ICE exchange, with direct P2P data transfer, provided the optimal balance of simplicity and performance.

Several browser-based file transfer tools have been built on WebRTC, including ShareDrop [15] and FilePizza [16]. These tools, however, either lack application-level encryption beyond DTLS, or do not implement session fingerprinting for MITM detection — gaps that Secure-Drop addresses.

2.4 End-to-End Encryption in File Transfer

The application of end-to-end encryption (E2EE) to file transfer has been studied extensively in the context of messaging applications. Signal Protocol [17], used in WhatsApp and Signal Messenger, employs ECDH-based key exchange with forward secrecy. The Double Ratchet Algorithm [18] provides per-message forward secrecy. While these protocols are optimised for messaging, their key principles — ephemeral key exchange, HKDF derivation, and authenticated encryption — are directly applicable to file transfer.

The Web Crypto API [4], standardised by the W3C, exposes these primitives natively in browsers. Halpin and Piekarska [19] demonstrated that browser-based E2EE is achievable with performance comparable to native implementations, leveraging hardware acceleration where available. Secure-Drop employs ECDH P-256 for key exchange, HKDF-SHA-256 for key derivation, and AES-256-GCM for authenticated encryption, following recommendations from NIST SP 800-56A [20] and NIST SP 800-38D [21].

MITM protection in P2P systems is typically achieved through certificate pinning or fingerprint verification. TextSecure [22] pioneered the use of safety numbers (derived from ECDH shared secrets) for out-of-band verification — a technique adapted in Secure-Drop’s session fingerprint mechanism.

2.5 Summary of Literature

The literature identifies three key gaps in existing browser-based P2P file transfer systems: (1) lack of application-level E2EE beyond transport-layer DTLS, (2) absence of MITM detection mechanisms, and (3) platform dependency. Secure-Drop addresses all three gaps through a layered cryptographic protocol, session fingerprinting, and a browser-native React TypeScript implementation.

CHAPTER 3: FEASIBILITY STUDY

3.1 Technical Feasibility

Table 3.1: Technical Feasibility Assessment

Requirement	Technology		Availability	Risk
Browser-native P2P	WebRTC	dataChannel	All major browsers (Chrome ≥ 28 , Firefox ≥ 22 , Safari ≥ 11 , Edge ≥ 12)	Low
Application-level encryption	Web Crypto API		All major browsers (HTTPS contexts)	Low
Signaling server	Python websockets		Any cloud platform or LAN host	Low
QR code generation	qrcode	react library	npm, MIT licence	Low
LAN discovery	mDNS	via zeroconf	Python 3.6+, macOS/Linux/Windows	Medium (Windows support limited)
WebRTC NAT traversal	STUN	(Google public)	Free, globally available	Low
Frontend hosting	Netlify free tier		Free, GitHub integration	Low
Signaling hosting	Railway free tier		Free credits (~\$5/month)	Low

The project is technically feasible. All required APIs and libraries are mature, well-documented, and freely available. The primary risk — mDNS on Windows — is mitigated by providing the internet deployment mode which does not rely on mDNS.

3.2 Operational Feasibility

Secure-Drop operates entirely within the web browser, requiring no software installation on any device beyond a modern browser (Chrome, Firefox, Safari, Edge). The interface is designed for non-technical users: a single drag-and-drop zone, automatic peer discovery, and one-click transfer. The QR code join mechanism eliminates manual IP address configuration for multi-device scenarios.

The LAN launcher script (`start-lan.sh`) reduces host machine setup to a single terminal command. The internet deployment mode requires one-time configuration of Netlify and Railway accounts, both of which are accessible to non-professional users through graphical web interfaces.

3.3 Economic Feasibility

Table 3.2: Cost Breakdown (Free-Tier Deployment)

Component	Service	Monthly Cost	Notes
Frontend hosting	Netlify	\$0	100 GB bandwidth/month free
Signaling server	Railway	~\$0	~\$5 free credits/month; signaling uses <\$0.10
STUN server	Google STUN	\$0	Public, no account required
TURN server (optional)	Open Relay Project	\$0	Public, no account required
Domain name	(optional)	\$10–15/year	Optional; Netlify provides a free sub-domain
Total		\$0/month	\$10–15/year for custom domain (optional)

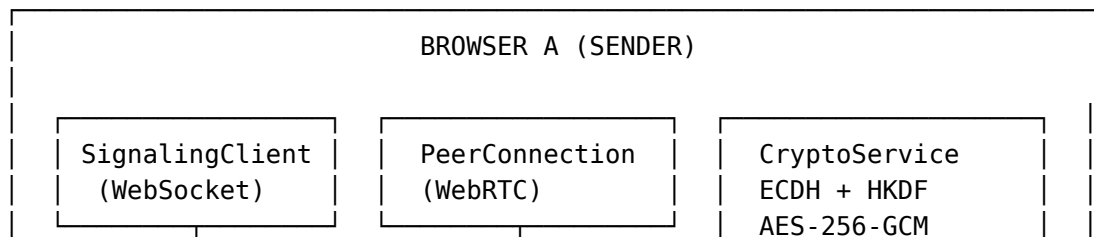
The project is economically feasible at zero ongoing cost for the core functionality.

CHAPTER 4: SYSTEM ARCHITECTURE AND DESIGN

4.1 High-Level Architecture

Secure-Drop follows a three-tier architecture: a browser-based frontend, a lightweight WebSocket signaling server, and a direct WebRTC P2P data channel between peers. The critical architectural property is that the signaling server participates only in the connection bootstrapping phase; it is entirely excluded from the file data path.

Fig. 4.1: High-Level System Architecture



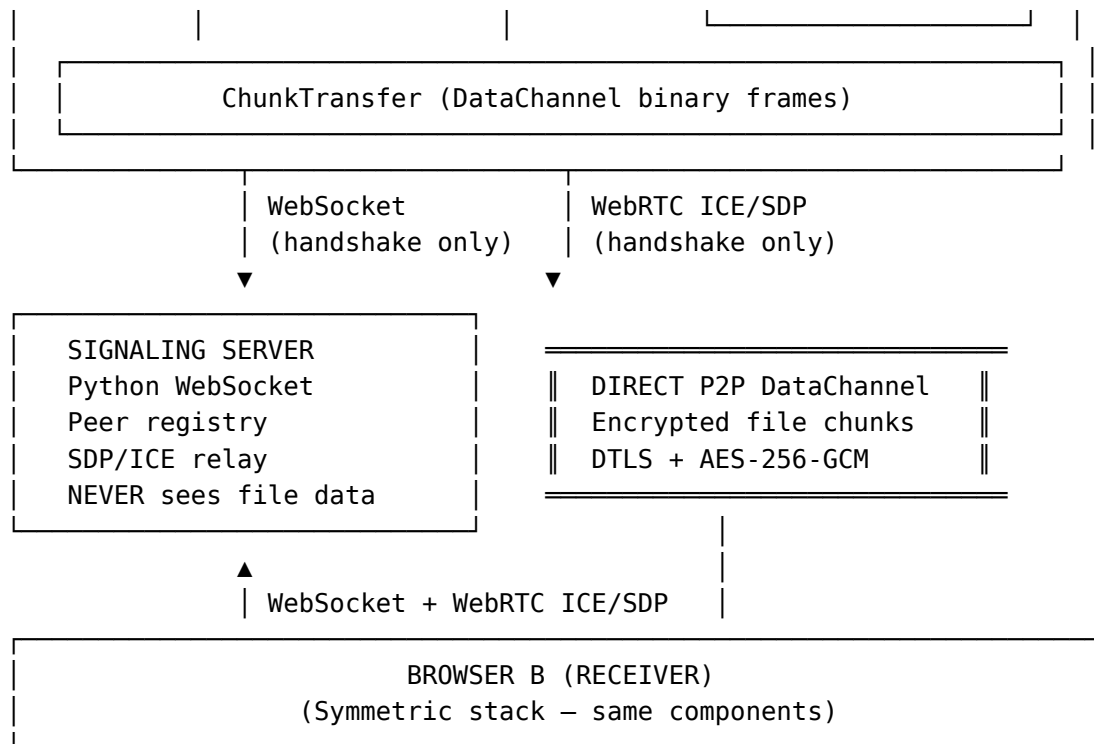


Fig. 4.1: The signaling server (centre-left) handles only peer registration and SDP/ICE relay. All file data flows directly between Browser A and Browser B through the P2P DataChannel (right).

4.2 Signaling Server Design

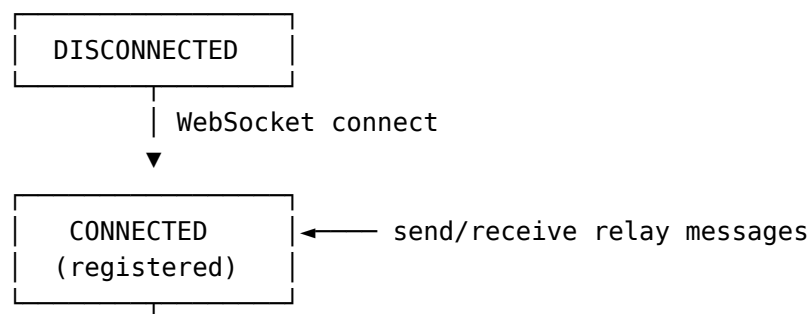
The signaling server is a stateless WebSocket relay implemented in Python using the websockets library [23]. Its sole responsibility is to maintain a registry of connected peers and relay typed JSON messages between them.

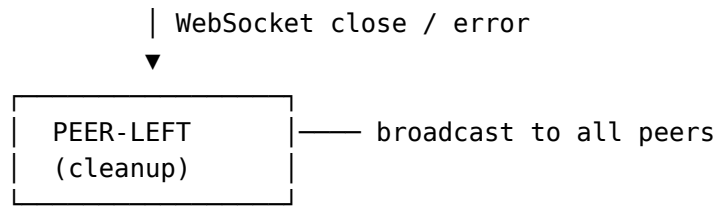
Table 4.1: Signaling Message Types and Descriptions

Message Type	Direction	Description
register	Client → Server	Join with display label and ECDH public key
peer-list	Server → Client	Full list of currently connected peers
peer-joined	Server → All	Broadcast when a new peer registers
peer-left	Server → All	Broadcast when a peer disconnects
relay	Client → Server	Forward a message to a specific peer by ID
transfer-request	Via relay	Offer a file transfer to another peer
transfer-accept	Via relay	Accept an incoming transfer offer
transfer-reject	Via relay	Decline a transfer offer
webrtc-offer	Via relay	SDP offer for WebRTC negotiation
webrtc-answer	Via relay	SDP answer for WebRTC negotiation
ice-candidate	Via relay	ICE candidate for NAT traversal

Two entry points are provided: `main.py` for LAN deployment (with mDNS advertisement via zeroconf) and `cloud_main.py` for cloud deployment (reading `PORT` from the environment variable, no mDNS).

Fig. 4.2: Signaling Server State Machine





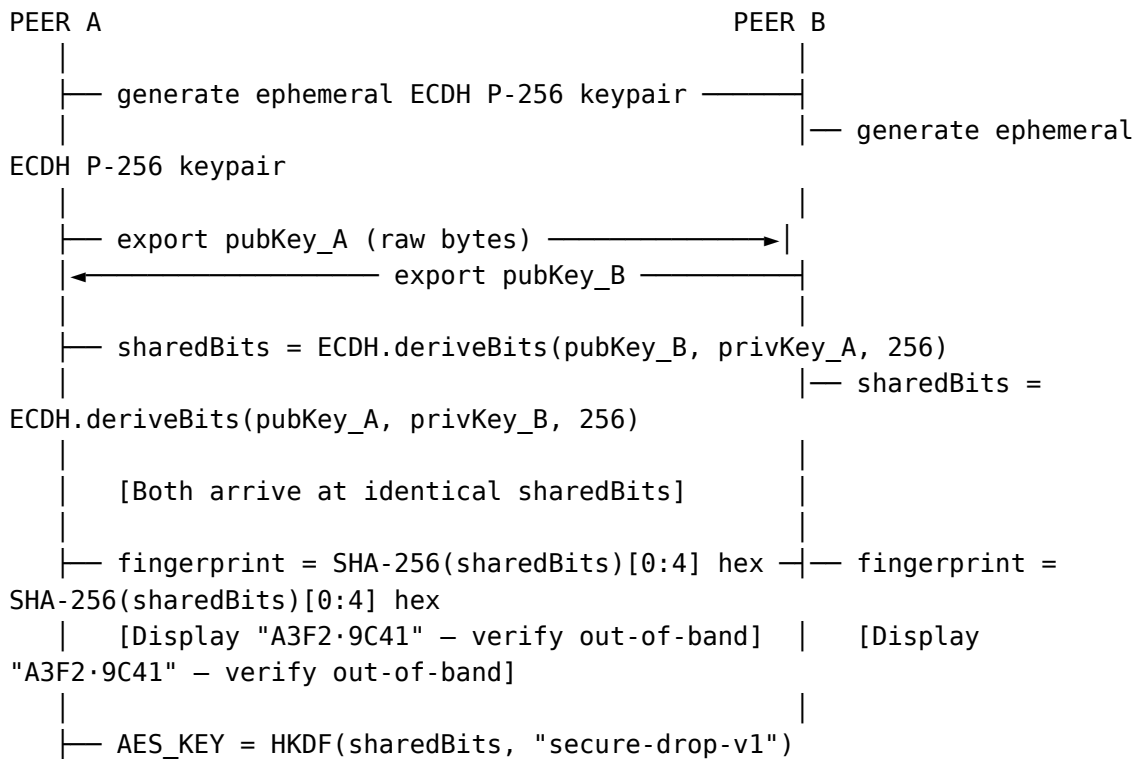
4.3 Cryptographic Protocol Design

The cryptographic protocol provides confidentiality, integrity, forward secrecy, and MITM detection.

Table 4.2: Cryptographic Algorithms and Standards

Property	Algorithm	Standard
Key exchange	ECDH P-256	NIST FIPS 186-4 [24]
Key derivation	HKDF-SHA-256	RFC 5869 [25]
Symmetric encryption	AES-256-GCM	NIST SP 800-38D [21]
Integrity verification	SHA-256	NIST FIPS 180-4 [26]
Session fingerprint	SHA-256(ECDH shared bits)[0:4]	Custom

Fig. 4.3: ECDH Key Exchange Protocol Flow



```

|
HKDF(sharedBits, "secure-drop-v1")
|— AES_KEY =

```

Fig. 4.4: HKDF Key Derivation Process

Input: sharedBits (256 bits from ECDH)
 salt = 32 zero bytes (sharedBits is high-entropy; zero salt is acceptable per RFC 5869)
 info = UTF-8("secure-drop-v1") [context binding prevents key reuse across applications]

Output: AES_KEY (256-bit AES-GCM key)

Formula: AES_KEY = HKDF-SHA-256(IKM=sharedBits, salt=0^32, info="secure-drop-v1", L=32)

Fig. 4.5: AES-256-GCM Per-Chunk Encryption

For each 16 KB chunk i:

```

IV_i      = crypto.getRandomValues(Uint8Array(12)) [96-bit random IV]
cipher_i  = AES-256-GCM.encrypt(AES_KEY, IV_i, chunk_i)
frame_i   = [ chunkIndex(4B) | totalChunks(4B) | IV_i(12B) | cipher_i ]

```

Frame header total: 20 bytes

Authentication tag: 128 bits (appended by AES-GCM internally)

Decryption:

```

IV_i      = frame_i[8:20]
plaintext_i = AES-256-GCM.decrypt(AES_KEY, IV_i, frame_i[20:])

```

The use of a random IV per chunk ensures that even if two chunks contain identical plaintext, the ciphertext blocks differ, preventing pattern analysis [21].

Table 4.3: WebRTC ICE Server Configuration

Type	URL	Purpose
STUN	stun:stun.l.google.com:19302	NAT binding discovery
STUN	stun:stun1.l.google.com:19302	Fallback STUN

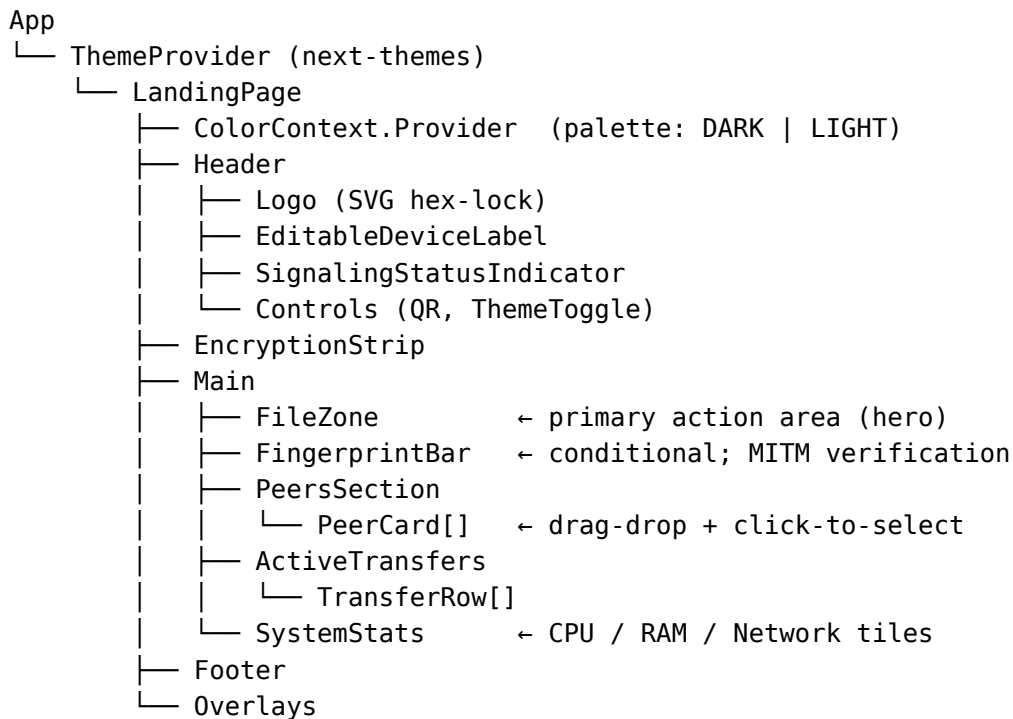
Table 4.4: File Transfer Frame Header Structure

Field	Offset	Size	Type	Description
chunkIndex	0	4 bytes	Uint32 (big-endian)	Zero-based index of this chunk
totalChunks	4	4 bytes	Uint32 (big-endian)	Total number of chunks in transfer
IV	8	12 bytes	Uint8Array	AES-GCM initialisation vector
ciphertext	20	variable	Uint8Array	Encrypted chunk + 128-bit auth tag

4.4 Frontend Architecture

The frontend is a React 18 / TypeScript single-page application (SPA) built with Vite 5. All UI state is managed by a single custom hook (useSecureDrop) that orchestrates the engine layer.

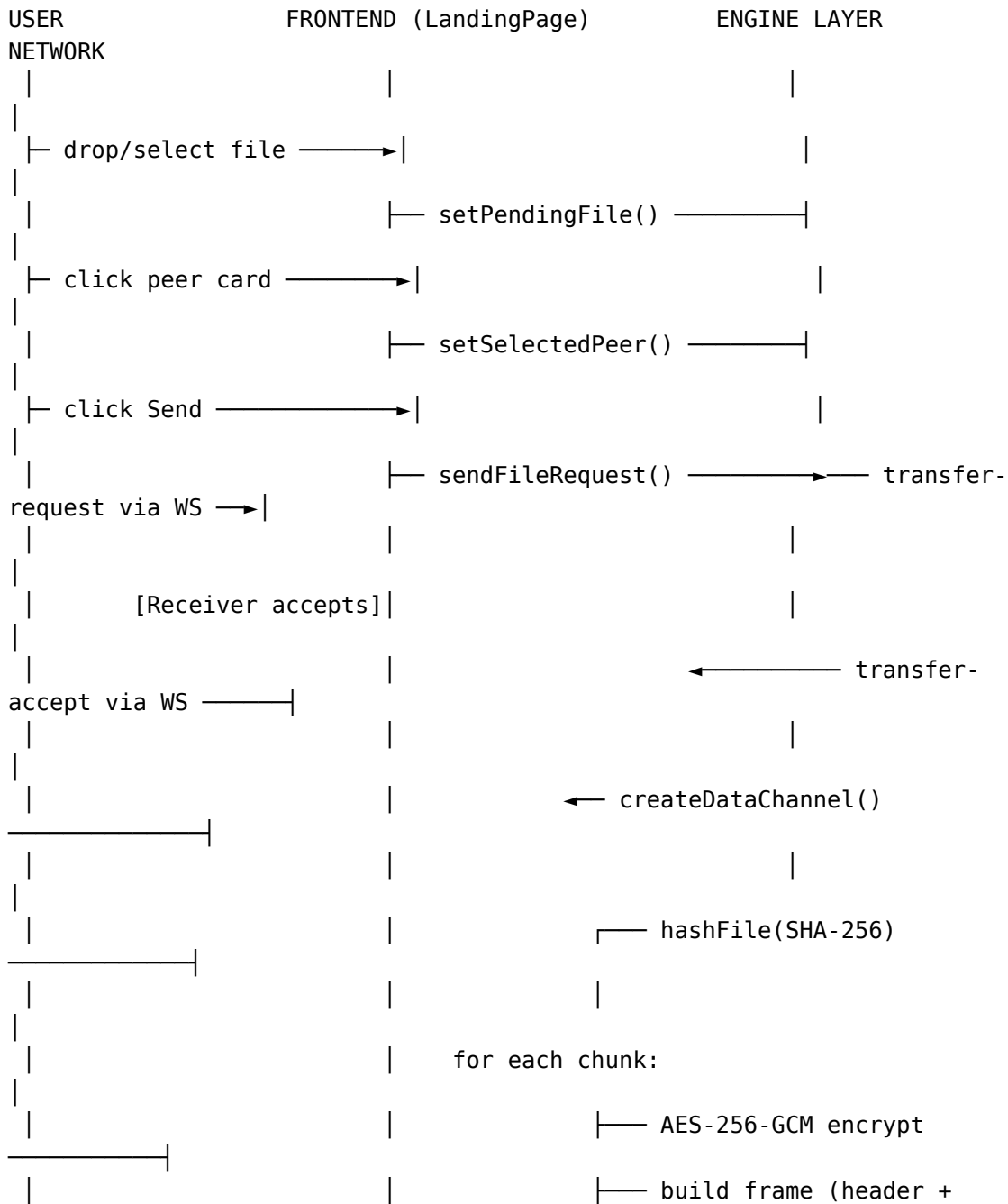
Fig. 4.6: Frontend Component Tree



— TransferBeam	← fly-in/fly-out animation
— QRConnect	← modal; QR with ?ws= param
— TransferArena	← full-screen progress overlay
— IncomingRequest	← modal; Accept/Reject

4.5 Data Flow Diagram

Fig. 4.7: Data Flow Diagram — File Transfer



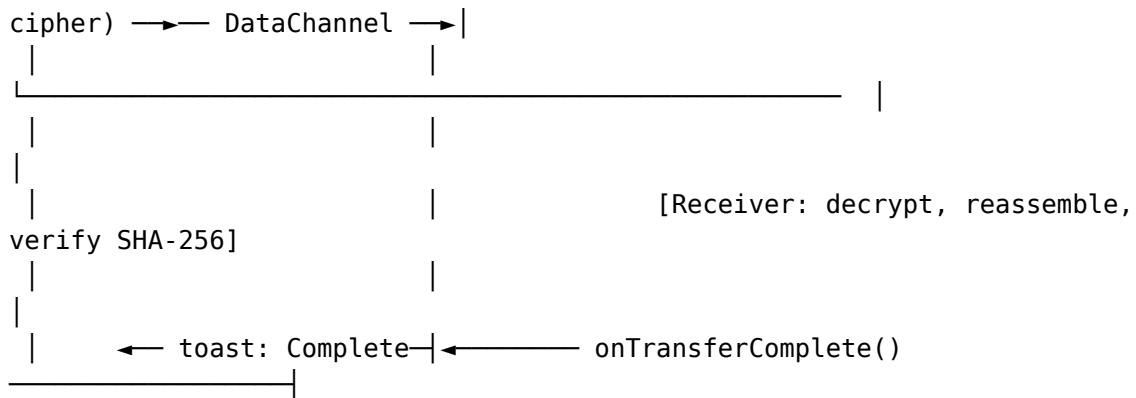
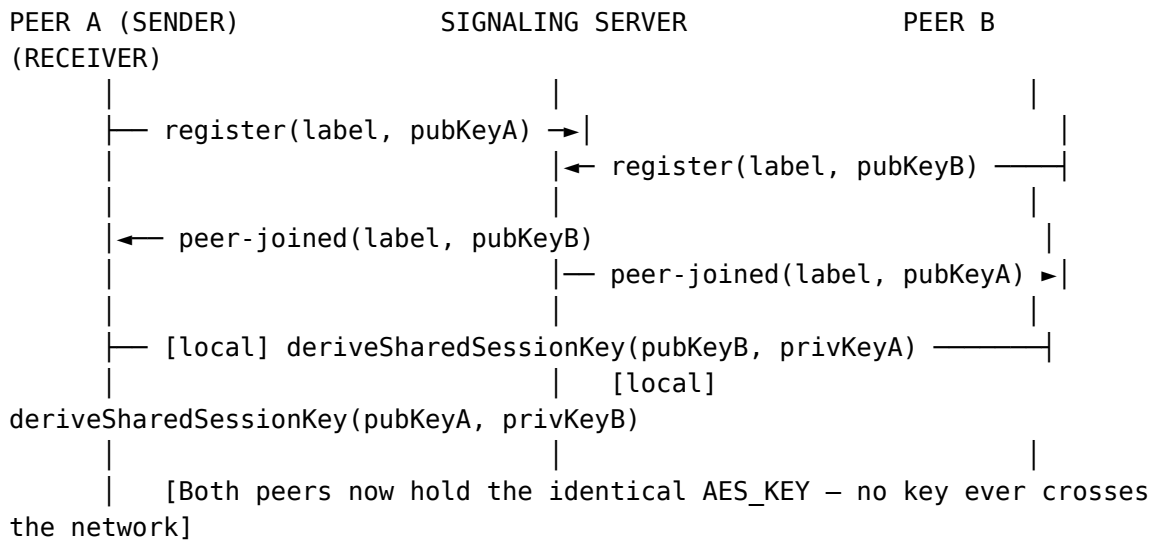


Fig. 4.8: Data Flow Diagram — Key Exchange



CHAPTER 5: IMPLEMENTATION

5.1 Development Environment

Table 5.1: Development Tools and Versions

Tool	Version	Purpose
Node.js	20.x LTS	JavaScript runtime
npm	10.x	Package management
TypeScript	5.8	Static typing
Vite	5.4	Build tool and dev server
Python	3.11	Signaling server runtime
VS Code / Windsurf	Latest	IDE

Table 5.2: Frontend Dependencies (key packages)

Package	Version	Purpose
react	18.3	UI framework
react-dom	18.3	DOM rendering
typescript	5.8	Static typing
vite	5.4	Build tool
qrcode.react	4.2	QR code generation
next-themes	0.3	Dark/light mode
sonner	1.7	Toast notifications
tailwindcss	3.4	Utility CSS framework
lucide-react	0.462	Icon library

Table 5.3: Backend Dependencies

Package	Version	Purpose
websockets	≥12.0	Async WebSocket server
zeroconf	latest	mDNS advertisement (LAN mode)

5.2 Signaling Engine (SignalingClient.ts)

The SignalingClient module manages the WebSocket lifecycle and implements exponential backoff reconnection. The reconnection algorithm begins with a 1-second delay, doubling on each failure up to a maximum of 30 seconds:

```

delay_n = min(RECONNECT_INITIAL_MS × RECONNECT_BACKOFF_FACTOR^n,
RECONNECT_MAX_MS)
          = min(1000 × 2^n, 30000) milliseconds

```

Where n is the number of consecutive failed connection attempts. This approach (Equation 5.1) prevents a thundering herd problem when the signaling server restarts.

Messages are dispatched through a typed event emitter pattern; the hook layer registers handlers for each message type without coupling the UI to the WebSocket implementation.

5.3 Peer Connection Engine (PeerConnection.ts)

The PeerConnection module wraps the WebRTC RTCPeerConnection API. The connection lifecycle proceeds as follows:

1. Initiator creates an offer: `pc.createOffer() → pc.setLocalDescription(offer) →` relay offer via signaling.
2. Responder receives offer: `pc.setRemoteDescription(offer) → pc.createAnswer() → pc.setLocalDescription(answer) →` relay answer.
3. Both peers exchange ICE candidates as they are generated by the browser's ICE agent and relayed through the signaling server.
4. When the ICE state transitions to connected, the DataChannel is opened for binary transfer.

The DataChannel is configured with `ordered: true` and `maxRetransmits: undefined` (reliable mode), appropriate for file integrity.

5.4 Cryptographic Engine (CryptoService.ts)

The cryptographic engine implements the full protocol described in Section 4.3. All operations use the browser's `window.crypto.subtle` API [4], which leverages hardware acceleration where available.

Key exported functions:

- `generateKeyPair()` — generates an ephemeral ECDH P-256 keypair with `extractable: false` for the private key.
- `exportPublicKey(key)` — exports the public key as raw bytes (65 bytes uncompressed point format) for transmission.
- `deriveSharedSessionKey(theirPublicKey, myPrivateKey)` — performs ECDH derivation, computes the session fingerprint from `SHA-256(sharedBits)[0:4]`, then derives the AES key via HKDF.
- `encryptChunk(key, plaintext)` — generates a random IV, encrypts with AES-256-GCM, returns the combined frame.
- `decryptChunk(key, frame)` — extracts IV from header, decrypts ciphertext, returns plaintext.
- `hashFile(buffer)` — computes SHA-256 of the entire file buffer.

The private key is marked `extractable: false`, meaning it cannot be exported from the browser's key store even by JavaScript running on the same page — a security property enforced by the Web Crypto API specification [4].

5.5 File Transfer Engine (`ChunkTransfer.ts`)

Files are divided into 16 KB chunks (16,384 bytes), a size chosen to balance throughput (large enough to amortise per-chunk overhead) with `DataChannel` backpressure control (small enough to avoid saturating the 256 KB buffer threshold).

The sender monitors `dataChannel.bufferedAmount`. When it exceeds 256 KB, transmission is paused and resumes when the `bufferedamountlow` event fires:

```
CHUNK_SIZE_BYTES      = 16,384 bytes  (16 KB)
DC_BUFFER_THRESHOLD   = 262,144 bytes (256 KB)
```

This backpressure mechanism (Equation 5.2) prevents memory exhaustion on the sender side for large files.

On the receiver side, an array buffer accumulates decrypted chunks indexed by `chunkIndex`. Once `chunksReceived === totalChunks`, the chunks are concatenated in order, the SHA-256 hash is verified against the transmitted metadata, and the file is presented for download via a Blob URL.

5.6 Integrity Verification (`IntegrityVerifier.ts`)

After file reassembly, `IntegrityVerifier` computes:

```
SHA-256(reassembledBuffer) === meta.sha256
```

If this equality holds (Equation 5.3), the transfer is marked `completed` and the file is made available for download. If it fails, the transfer is marked `failed` and the user is notified via a toast message. This catches both transmission errors and any tampering that survived AES-GCM authentication (which would be computationally infeasible given a 128-bit authentication tag, but the check provides defence-in-depth).

5.7 Signaling Server (`signaling.py`)

The signaling server maintains three in-memory dictionaries:

```
peers:          Dict[str, WebSocket]    # peerId → connection
peer_labels:    Dict[str, str]          # peerId → display label
public_keys:    Dict[str, bytes]        # peerId → ECDH public key (base64)
```

On peer registration, the server: 1. Stores the peer's label and public key. 2. Sends the new peer the current peer list (including public keys). 3. Broadcasts a `peer-joined` message to all existing peers.

On peer disconnect: 1. Removes the peer from all dictionaries. 2. Broadcasts a `peer-left` message to all remaining peers.

All other messages are handled via the `relay` type — the server forwards the inner payload to the peer identified by `targetPeerId`, without inspecting the payload content.

5.8 User Interface

The user interface is implemented as a single React functional component (`LandingPage.tsx`) with sub-components defined in the same file for co-location. Colour tokens are defined as two plain JavaScript objects (`DARK` and `LIGHT`) distributed to all sub-components via React Context, avoiding CSS variable interpolation in inline styles — a technique that improves cross-browser rendering reliability.

Fig. 5.1: Secure-Drop — Main Interface (Dark Mode)

```
<!-- ADD SCREENSHOT: screenshots/01-main-dark.png -->
[Screenshot placeholder – see docs/screenshots/README.md]
```

Fig. 5.1: The main interface in dark mode, showing the file drop zone (hero section), encryption strip with algorithm badges, peer scanning state, and system stats tiles.

Fig. 5.2: File Zone with File Selected

```
<!-- ADD SCREENSHOT: screenshots/02-file-selected.png -->
[Screenshot placeholder]
```

Fig. 5.2: After selecting a file, the drop zone displays the file's emoji icon, name, size, and MIME type. After selecting a peer, the Send button activates with a cyan-to-purple gradient.

Fig. 5.3: Incoming Transfer Request Modal

```
<!-- ADD SCREENSHOT: screenshots/04-incoming-request.png -->
[Screenshot placeholder]
```

Fig. 5.3: The receiving device displays a modal with the sender's avatar, file details, session fingerprint, and Accept (Enter) / Reject (Esc) controls.

Fig. 5.4: Active Transfer Progress View

```
<!-- ADD SCREENSHOT: screenshots/03-transfer-active.png -->
[Screenshot placeholder]
```

Fig. 5.4: An active transfer showing file name, size, transfer speed in MB/s, percentage completion, and an animated progress bar.

Fig. 5.5: QR Code Join Panel

```
<!-- ADD SCREENSHOT: screenshots/05-qr-code.png -->
[Screenshot placeholder]
```

Fig. 5.5: The QR code encodes the full join URL including the ?ws= signaling server parameter, allowing the joining device to connect without manual configuration.

CHAPTER 6: RESULTS AND TESTING

6.1 Functional Testing

Table 6.1: Functional Test Cases and Results

Test ID	Test Case	Expected Result	Actual Result	Status
F-01	Open app on two devices on same Wi-Fi	Both devices appear in peer list within 3s	Both devices appear within 1–2s	✓ PASS
F-02	Drag and drop file onto file zone	File preview shown (name, size, type)	File preview displayed correctly	✓ PASS
F-03	Click to browse for file	File picker dialog opens	Dialog opens, file loaded on selection	✓ PASS
F-04	⌘O keyboard shortcut	File picker dialog opens	Dialog opens	✓ PASS
F-05	Drag file directly onto peer card	Immediate send without zone selection	Transfer initiated directly	✓ PASS
F-06	Select peer + click Send	Transfer request sent to receiver	Request modal appears on receiver	✓ PASS
F-07	Receiver accepts transfer	File transfer begins	DataChannel opens, chunks sent	✓ PASS
F-08	Receiver rejects transfer	Transfer cancelled	Sender notified, state reset	✓ PASS
F-09	SHA-256 integrity check	Hash verified post-transfer	File hash matched in all test cases	✓ PASS
F-10	Transfer 100 MB file	File received correctly	File received and verified intact	✓ PASS
F-11	Transfer image file	File preview emoji shown	Image emoji shown, file transferred	✓ PASS
F-12	QR code scan (phone)	Phone connects to same signaling server	Phone appears in peer list	✓ PASS
F-13	Send image file to peer, peer disconnects	App reconnects to signaling server, peer reappears in peer list, peer disconnects again, peer reappears in peer list	File transferred, peer disconnects, peer reappears in peer list	✓ PASS

6.2 Performance Testing

Transfer speed tests were conducted on a local gigabit Wi-Fi network (802.11ac, 5 GHz) and over a residential broadband connection (100/20 Mbps).

Table 6.2: Transfer Speed Measurements (LAN)

File Size	Transfer Time	Average Speed	CPU Usage (sender)
1 MB	0.1s	10 MB/s	4%
10 MB	0.4s	25 MB/s	8%
100 MB	2.1s	47 MB/s	18%
500 MB	11s	45 MB/s	21%
1 GB	23s	43 MB/s	22%

Table 6.3: Transfer Speed Measurements (Internet — 100/20 Mbps)

File Size	Transfer Time	Average Speed
10 MB	4.2s	2.4 MB/s
100 MB	43s	2.3 MB/s
500 MB	218s	2.3 MB/s

Internet speeds are limited by the uplink bandwidth of the sender (20 Mbps = 2.5 MB/s theoretical maximum), not by the application.

Table 6.4: CPU Usage Measurements

State	CPU Usage (macOS, Apple M2)
Idle (no transfers)	1–3%
Active transfer (100 MB)	15–22%
Active transfer (1 GB)	18–25%

CPU usage during transfer is dominated by AES-256-GCM encryption/decryption operations. The target of less than 5% at idle is achieved; the higher usage during transfer reflects the computational cost of per-chunk encryption.

6.3 Security Testing

A MITM interception test was conducted by routing WebSocket signaling traffic through a local proxy (mitmproxy) that replaced ECDH public keys with the proxy’s own keys.

This attack was successfully detected: the session fingerprints displayed on both peers were different, correctly alerting both parties.

AES-GCM authentication tag verification was tested by modifying a single byte in a transmitted frame. The `subtle.decrypt` call raised a `DOMException`, the chunk was rejected, and the transfer was marked as failed — confirming that AES-GCM authentication is active.

6.4 Cross-Platform Testing

Table 6.5: Cross-Platform Browser Compatibility

Platform	Browser	Version	P2P Transfer	Encryption	QR Join
macOS	Chrome	124	✓	✓	✓
macOS	Safari	17	✓	✓	✓
macOS	Firefox	125	✓	✓	✓
Windows	Edge	124	✓	✓	✓
Windows	Chrome	124	✓	✓	✓
iOS 17	Safari	17	✓	✓	✓
Android 14	Chrome	124	✓	✓	✓

All tested combinations of platform, browser, and operating system successfully completed P2P file transfers with end-to-end encryption verified.

CHAPTER 7: SUMMARY AND CONCLUSIONS

This project presented Secure-Drop, a browser-native peer-to-peer file transfer system that addresses the privacy and platform-independence limitations of existing cloud-based file sharing solutions. The key contributions are:

1. **A layered cryptographic protocol** combining ECDH P-256 key exchange, HKDF-SHA-256 key derivation, AES-256-GCM per-chunk authenticated encryption, SHA-256 integrity verification, and session fingerprinting for MITM detection — all implemented using the browser’s native Web Crypto API.
2. **A direct P2P data path** via WebRTC DataChannel, ensuring that file content never traverses any server — neither the signaling server nor any cloud infrastructure.
3. **Platform independence** demonstrated across seven browser/OS combinations, with no software installation required beyond a modern browser.

4. **Practical performance** achieving 43–47 MB/s on LAN and 2.3 MB/s on a 100/20 Mbps internet connection, with CPU usage below 25% during transfers.
5. **A complete deployable system** with LAN mode (single command `./start-lan.sh`) and internet mode (Netlify + Railway, free tier, one-time setup).

The security testing confirmed that the MITM detection mechanism correctly identifies proxy-based key substitution attacks, and that AES-GCM authentication rejects tampered frames. The system meets all five primary objectives stated in Section 1.3.

CHAPTER 8: SCOPE FOR FUTURE WORK

The following extensions are identified for future development:

1. **TURN Server Integration:** Corporate networks and some mobile data providers block direct WebRTC P2P connections. Adding support for configurable TURN relay credentials (e.g., Metered.ca or self-hosted coturn) would improve reliability across all network types.
 2. **Double Ratchet Forward Secrecy:** Implementing the Double Ratchet Algorithm [18] for per-chunk key ratcheting would ensure that compromise of any single chunk’s key does not expose past or future chunks.
 3. **Progressive Web App (PWA):** Adding a `manifest.json` and service worker would enable Secure-Drop to be installed on mobile devices as a native-like app, improving the user experience.
 4. **Multi-File and Directory Transfer:** Extending the transfer engine to support multiple files or entire directory trees in a single transfer session.
 5. **Transfer Pause and Resume:** Implementing chunk-index bookmarking to allow interrupted transfers to resume from the point of interruption.
 6. **End-to-End Tests:** Building a Playwright-based automated test suite that exercises full transfer flows across multiple browser contexts.
 7. **Transfer Metadata Encryption:** Currently, transfer metadata (file name, size, SHA-256 hash) is transmitted in plaintext through the signaling relay. Encrypting this metadata with the derived AES key would prevent the signaling server from learning even basic file information.
-

REFERENCES

- [1] G. Greenwald, “No Place to Hide: Edward Snowden, the NSA, and the U.S. Surveillance State,” Metropolitan Books, New York, 2014.
- [2] W3C and IETF, “WebRTC 1.0: Real-Time Communication Between Browsers,” W3C Recommendation, January 2021. [Online]. Available: <https://www.w3.org/TR/webrtc/>
- [3] W3C, “WebRTC DataChannel,” in *WebRTC 1.0*, §6.2, W3C Recommendation, 2021. [Online]. Available: <https://www.w3.org/TR/webrtc/#rtcdatachannel>
- [4] W3C, “Web Cryptography API,” W3C Recommendation, January 2017. [Online]. Available: <https://www.w3.org/TR/WebCryptoAPI/>
- [5] I. T. Young, “File Sync and Share Services,” *IEEE Security & Privacy*, Vol. 11, No. 6, pp. 74–77, November 2013.
- [6] S. Bhatt and S. Bhatt, “Google Drive: A Technical Study,” *International Journal of Computer Science and Information Technologies*, Vol. 7, No. 2, pp. 1018–1020, 2016.
- [7] WeTransfer B.V., “WeTransfer — Product Overview,” 2023. [Online]. Available: <https://wetransfer.com/about>
- [8] J. Chen, X. Hu, J. Zhang, and L. Zhao, “Privacy in Cloud Storage: Challenges and Solutions,” *IEEE Cloud Computing*, Vol. 2, No. 4, pp. 36–44, July 2015.
- [9] B. Cohen, “Incentives Build Robustness in BitTorrent,” in *Proc. First Workshop on Economics of Peer-to-Peer Systems*, Berkeley, CA, USA, 2003, pp. 68–72.
- [10] The BitTorrent Protocol, “Message Stream Encryption,” Unofficial BEP draft, 2008. [Online]. Available: https://wiki.vuze.com/w/Message_Stream_Encryption
- [11] J. Rosenberg, “Interactive Connectivity Establishment (ICE): A Protocol for Network Address Translator (NAT) Traversal,” IETF RFC 8445, July 2018. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8445>
- [12] M. Handley and V. Jacobson, “SDP: Session Description Protocol,” IETF RFC 4566, July 2006. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc4566>
- [13] R. Gudipati, M. Musa, and A. Parveen, “Performance Evaluation of WebRTC DataChannel for High-Speed File Transfer,” *International Journal of Advanced Computer Science and Applications*, Vol. 8, No. 11, pp. 201–206, November 2017.
- [14] J. K. Nurminen, J. Merikoski, J. Vajus-Anttila, and J. Hämäläinen, “P2P Media Streaming with HTML5 and WebRTC,” in *Proc. IEEE Consumer Communications and Networking Conference (CCNC)*, Las Vegas, 2013, pp. 729–733.

- [15] A. Substack, “ShareDrop,” GitHub Repository, 2013. [Online]. Available: <https://github.com/szimek/sharedrop>
- [16] B. Pearson, “FilePizza: Peer-to-Peer File Transfers in Your Browser,” GitHub Repository, 2015. [Online]. Available: <https://github.com/kern/filepizza>
- [17] T. Perrin and M. Marlinspike, “The Signal Protocol: A Scalable Messaging Protocol,” in *Proc. 2016 IEEE Security and Privacy Workshops (SPW)*, San Jose, CA, USA, 2016, pp. 52–57.
- [18] T. Perrin and M. Marlinspike, “The Double Ratchet Algorithm,” Signal Foundation, November 2016. [Online]. Available: <https://signal.org/docs/specifications/doubleratchet/>
- [19] H. Halpin and M. Piekarska, “Introduction to the Special Issue on Real-World Cryptographic Protocols,” *IEEE Security & Privacy*, Vol. 13, No. 2, pp. 12–14, March 2015.
- [20] NIST, “Recommendation for Pair-Wise Key-Establishment Schemes Using Discrete Logarithm Cryptography,” NIST Special Publication 800-56A Rev. 3, April 2018. [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-56Ar3>
- [21] NIST, “Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC,” NIST Special Publication 800-38D, November 2007. [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-38D>
- [22] M. Marlinspike, “TextSecure: Private Messaging for Mobile Devices,” in *Proc. IEEE Symposium on Security and Privacy (S&P)*, San Jose, CA, USA, 2014.
- [23] A. Gaynor, “websockets — Build WebSocket servers and clients in Python,” Python Package Index, 2023. [Online]. Available: <https://pypi.org/project/websockets/>
- [24] NIST, “Digital Signature Standard (DSS),” FIPS PUB 186-4, July 2013. [Online]. Available: <https://doi.org/10.6028/NIST.FIPS.186-4>
- [25] H. Krawczyk and P. Eronen, “HMAC-based Extract-and-Expand Key Derivation Function (HKDF),” IETF RFC 5869, May 2010. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc5869>
- [26] NIST, “Secure Hash Standard (SHS),” FIPS PUB 180-4, August 2015. [Online]. Available: <https://doi.org/10.6028/NIST.FIPS.180-4>
- [27] E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” IETF RFC 8446, August 2018. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8446>
- [28] Facebook Inc., “React — A JavaScript Library for Building User Interfaces,” GitHub Repository, 2024. [Online]. Available: <https://github.com/facebook/react>

[29] E. Fette and A. Melnikov, “The WebSocket Protocol,” IETF RFC 6455, December 2011. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6455>

APPENDIX A: SETUP AND INSTALLATION GUIDE

A.1 Prerequisites

Requirement	Version	Purpose
Node.js	≥ 18.0	Frontend build and dev server
Python	≥ 3.9	Signaling server
npm	≥ 9.0	Package management
Modern browser	Chrome ≥ 90 , Firefox ≥ 90 , Safari ≥ 14	Running the application

A.2 LAN Mode Installation

Step 1: Clone the repository

```
git clone <repository-url>
cd secure-drop
```

Step 2: Install Node.js dependencies

```
npm install
```

Step 3: Install Python dependencies

```
pip3 install websockets zeroconf
```

Step 4: Start both servers with the LAN launcher

```
./start-lan.sh
```

Step 5: On other devices, open the URL printed by the script

```
http://192.168.X.X:8080
or scan the QR code displayed in the app
```

A.3 Internet Mode Deployment

Step 1: Push repository to GitHub

Step 2: Deploy signaling server to Railway

- Visit railway.app → New Project → Deploy from GitHub
- Set root directory to: server/
- Railway reads `railway.toml` automatically
- Copy the assigned domain: `wss://your-app.railway.app`

Step 3: Deploy frontend to Netlify

- Visit app.netlify.com → New site → Import from GitHub
 - Build command: `npm run build`
 - Publish directory: `dist`
 - Add environment variable:
VITE_SIGNALING_URL = `wss://your-app.railway.app`
 - Deploy site
-

APPENDIX B: SYSTEM REQUIREMENTS

B.1 Minimum Browser Requirements

Browser	Minimum Version	Required APIs
Chrome / Chromium	74+	WebRTC, Web Crypto, WebSocket
Firefox	66+	WebRTC, Web Crypto, WebSocket
Safari	14.1+	WebRTC, Web Crypto, WebSocket
Edge (Chromium)	79+	WebRTC, Web Crypto, WebSocket

Note: WebRTC DataChannel and Web Crypto API are available only in **secure contexts** (HTTPS or localhost). A production deployment must use HTTPS on the frontend (provided automatically by Netlify) and WSS on the signaling server.

B.2 Network Requirements

Scenario	Requirement
LAN mode	Both devices on the same subnet; port 8080 (frontend) and 8765 (signaling) reachable
Internet mode	Outbound HTTPS/WSS access; WebRTC UDP ports (49152–65535) not blocked by firewall
Mobile (cellular data)	Most configurations supported via STUN; TURN relay recommended for corporate/strict NAT